Iterator

Iterator Pattern

Traverse without exposing internals

DEFINITION

Provide a way to access elements of a collection sequentially without exposing its underlying representation.

Iterator factors traversal out of the collection into a cursor that remembers "where am I". Callers pull one element at a time through a uniform interface, so the same loop works over an array, a tree, or a remote page — and the collection's internal representation stays hidden and free to change.

WHY IT MATTERS

When callers loop by reaching into a collection's internals, every caller breaks the day you swap an array for a tree or a paged API. A cursor decouples traversal from storage: the collection just offers "give me an iterator", callers walk it the same way regardless of shape, and you get lazy, one-at-a-time access that streams huge or infinite sequences in flat memory. Many cursors can walk one collection at once.

How it works

Iterator	The cursor interface — hasNext() / next() / current; owns the position.
ConcreteIterator	Holds the traversal state over one aggregate.
Aggregate	The collection that hands out cursors (Symbol.iterator / createIterator()).

HOW TO APPLY

1. Decide the traversal (forward, reverse, DFS / BFS) and what one "element" is.
2. Expose it as a cursor — in JS, implement [Symbol.iterator] with a generator.
3. Keep the position inside the cursor so multiple walks stay independent.
4. Add return() / finally to release resources (files, sockets) on early break.

> LITMUS TEST
>
> Are callers reaching into the collection's shape to walk it, or do you need lazy, multiple, or uniform traversal? If yes, hand out a cursor instead of the internals.

— Smell → fix

Callers read the backing array directly, so the representation leaks:

```
// callers must know it's an array inside
for (const n of tree._nodes) visit(n)
// swap _nodes to a linked list later
// → every call site breaks
```

↓ HAND OUT A CURSOR, HIDE THE STORAGE

```
class Tree {
*[Symbol.iterator]() {
yield this.value
for (const child of this.kids) yield* child
}
}
for (const n of tree) visit(n) // structure hidden
```

The generator IS the iterator: callers write for…of and never see kids vs a linked list. Swap the internal structure and every caller keeps working; each for…of gets its own fresh cursor.

USE WHEN

- ✓ Custom collections
- ✓ Uniform traversal; lazy or infinite sequences

AVOID WHEN

- ▲ Plain arrays with built-in iteration

Vasyl Krupka · Ukraine

— Trade-offs & pitfalls

PROS

+ Hides the structure
+ Multiple simultaneous traversals

CONS

− Overhead for simple cases

COMMON PITFALLS

▲ Mutating the collection mid-walk corrupts the cursor — fail-fast iterators throw (ConcurrentModificationException); some snapshot instead.

▲ A bare generator object is one-shot: it is exhausted after one pass. Implement Iterable so each for...of mints a fresh iterator.

▲ Sharing one iterator across consumers interleaves and corrupts the walk — the cursor carries position.

WHERE YOU SEE IT

- JS Symbol.iterator / for...of and function* generators; Node Symbol.asyncIterator with for await...of over streams.
- java.util.Iterator (hasNext / next) and C# IEnumerator / LINQ's lazy iterators.
- Database cursors and paginated APIs walked one page at a time.

SEE ALSO

Composite	an Iterator is the usual way to walk a Composite tree
Visitor	pairs with traversal to run an operation per element visited
Factory Method	the aggregate's createIterator() is a factory for the right cursor
Strategy	swapping the iterator swaps the traversal order without touching the collection

Interview & sources

External vs internal iterator?

External: the client calls next() and controls the loop. Internal: the collection runs the loop and calls you back (forEach).

What if you mutate mid-iteration?

Fail-fast iterators throw (ConcurrentModificationException); snapshot iterators walk a copy. JS just sees the live array.

Is a generator an iterator?

Yes — calling a function* returns an iterator (and iterable) with next(); it is one-shot until you call the factory again.

Why lazy iteration?

You process one element at a time, so you can handle infinite or huge sequences in constant memory.

REMEMBER

Hand out a cursor that walks the elements — callers traverse without ever seeing how the collection is stored.

SOURCES

GoF — "Design Patterns" (1994): Iterator (behavioral)

Freeman & Robson — "Head First Design Patterns" (2nd ed., 2020): Iterator & Composite